

DinnerPlanner: Development Log

Bhatta Mridula Chandrika

1 Overview

DinnerPlanner is a single-page web application for planning a dinner menu. The user searches an online recipe database, opens a dish to see its ingredients and price, and adds it to a menu. A sidebar shows the number of guests, the dishes on the menu and the total cost. The app is built with **React** (with a parallel **Vue** front end), **MobX** for reactivity, **React Router** for navigation and **Firebase Firestore** for persistence, and it is tested with **Vitest**.

The app was built step by step over three tutorial weeks, followed by bonus tasks. This report follows the commit history: for each week it lists what was built and what was learnt. It uses the *Model–View–Presenter* (MVP) pattern throughout. The **Model** holds the application state and knows nothing about the UI. **Views** only render props and fire events. **Presenters** connect the two.

Stage	Date	Commits
Setup	14 Jan	initial setup for web
Week 1	1 Feb	Exercise 1.1, menu price, TW1.2.15–TW1.2.30, TW1.3, TW1.4, TW1.5, bootstrapping error
Week 2	15 Feb	TW2.1.2–TW2.1.4, TW2.2.1–TW2.2.3, TW2.3.1, TW2.3.2, TW2.4.1, TW2.4.2, TW2.5, fixes
Week 3	28 Feb–1 Mar	TW3.1 and TW3.2, TW3.3, TW3 user improvements, comments
Bonus	28–29 May	update tests, Reactivity on persistence, Search-as-you-type with debouncing, Infinite search result scroll
Wrap-up	27 Sep	Merge user improvements into main, Remove embedded Firebase credentials

Table 1: Development timeline from the git history.

2 Week 1: Model, Views and Array Rendering

What was done.

- Exercise 1.1, menu price: wrote the model (`DinnerModel.js`) with `numberOfGuests`, `dishes`, `addToMenu`, `removeFromMenu` and input validation. Wrote utilities to compute the menu price and to sort dishes and ingredients.
- TW1.2.15: made the model reactive with a MobX observable (`mobxReactiveModel.js`) and exposed it as `myModel` in the Console.
- TW1.2.20–TW1.2.30: added the Summary and Sidebar presenters and views. The sidebar lists the dishes with their course type and price, and the summary lists all ingredients, grouped by aisle and scaled to the number of guests.
- TW1.3–TW1.5: added the `+/-` guest buttons and the remove (`x`) buttons as custom events, which the Sidebar presenter handles by calling the model.

What was learnt.

- The roles of Model, View and Presenter, and why the model must not know about the UI.
- *Array rendering*: mapping model arrays to JSX rows with `map`, `filter` and `sort`.
- *Custom events* and “props down, events up”: a view *fires* an event (e.g. `onNumberChange`), and the presenter *handles* it by changing the model.

-	5	+
☒	French toast	106.70
☒	Sourdough Starter starter	56.10
☒	Meat balls main course	412.05
☒	Chocolate Ice cream dessert	82.10
Total:		656.95
Summary for 5 persons:		
Name	Aisle	Quantity unit
water	(home)	400.00 ml
white bread	Bakery	10.00 slices
black chockolate	Baking	750.00 g
flour	Baking	75.00 g
ground nutmeg	Baking	2.50 g
seasoned bread crumbs	Baking	75.00 g
sugar	Baking	35.00 g
water activated dry yeast	Baking	2.50 g
grated Parmesan cheese	Cheese	25.00 g
egg	Eggs & Dairy	30.00 pcs
milk	Eggs & Dairy	250.00 ml
ice cream	Frozen	500.00 ml
extra lean ground beef	Meat	575.00 g
small onion, diced	Produce	1.25 pcs
Italian seasoning	Spices	2.50 g
Worcestershire sauce	Spices	30.00 ml
crushed red pepper flakes	Spices	2.50 g
dried oregano	Spices	2.50 g
garlic salt	Spices	2.50 g
sea salt	Spices	2.50 g

Figure 1: End of Week 1: dishes added from the Console using `myModel`, with the sidebar (top) and summary (bottom) for 5 guests.

- How a reactive object (MobX `observable` with `observer` components) re-renders the UI automatically when the model changes.

3 Week 2: Asynchronous Data and Promises

What was done.

- TW2.1.2–TW2.1.4: wrote `dishSource.js`, which calls the Spoonacular API through a proxy. `searchDishes` builds a query string with `URLSearchParams`, and `getDishDetails` fetches the details of one dish. The API key is sent in a request header.
- TW2.2.1–TW2.2.3: wrote `resolvePromise`, which stores `promise`, `data` and `error` in a *promise state* in the model. Added `searchResultsPromiseState`, `currentDishPromiseState` and a MobX `reaction` that fetches dish details as a *side effect* whenever `currentDishId` changes.
- TW2.3.1–TW2.4.2: added the Search and Details presenters and views, and `SuspenseView`, which shows a loading image while a promise is pending and an error message if it fails. An initial search runs on every page load.
- TW2.5, fixes: connected the result cards to the details view and the *Add to menu* button to the model.

What was learnt.

- *Fetch is a two-stage promise.* `fetch()` resolves as soon as the response headers arrive, so `response.ok` (the HTTP status) can be checked first. `response.json()` is a second promise, because the response body is still being downloaded. Both stages appear in `dishSource.js` (`responseACB` and `keepResultsACB`).
- *HTTP basics:* the method (GET), headers (API key), query string (`query`, `type`, `offset`) and response status, and how the API uses each of them.
- *Avoiding race conditions:* if the user searches twice quickly, the first response may arrive last. `resolvePromise` only stores a result if its promise is still the current one (if `(promiseState.promise !== prms) return;`).

- *Good UX for asynchronous data*: conditional rendering of “no data”, loading, error and data states.
- *Synchronous vs. asynchronous callbacks*, e.g. `map` runs its callback immediately, while `then` runs its callback later.

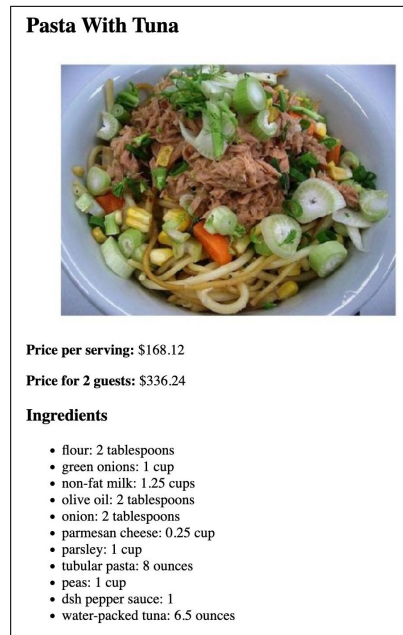


Figure 2: Details view: data from `currentDishPromiseState`, prices scaled to 2 guests.

4 Week 3: Persistence, Navigation and User Experience

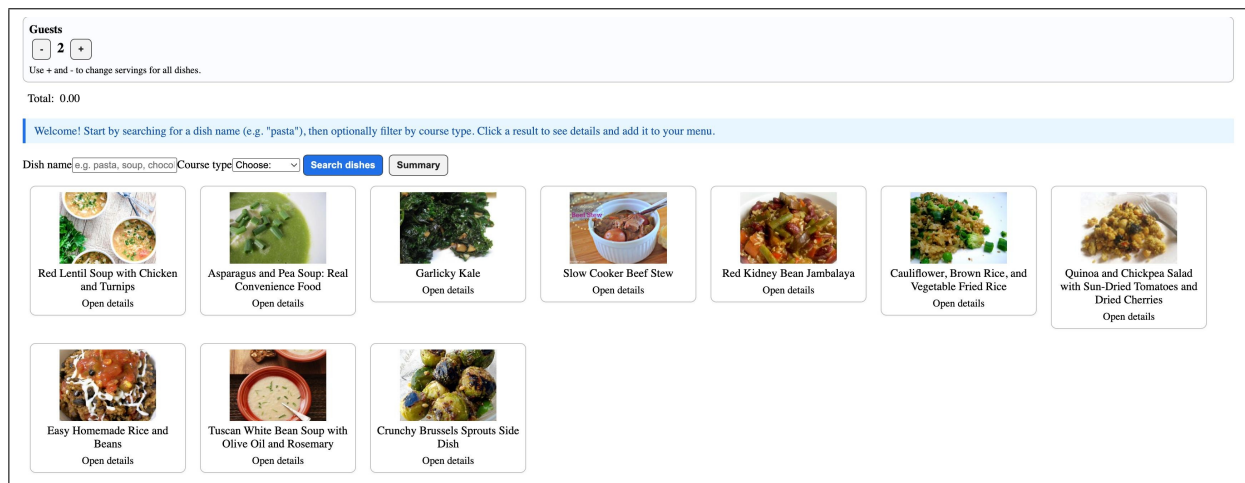
What was done.

- TW3.1 and TW3.2: added Firebase Firestore persistence (`firestoreModel.js`). At startup, `connectToPersistence` reads the guest count, menu and current dish. A reactive side effect then saves them whenever they change. While the data is loading, `model.ready` is `false` and a loading screen is shown, so that saving cannot overwrite the cloud data before it has been read.
- TW3.3: added navigation with a React Router hash router (`/search`, `/details`, `/summary`).
- TW3 user improvements, comments: ran a think-aloud test and improved the UI (see below).

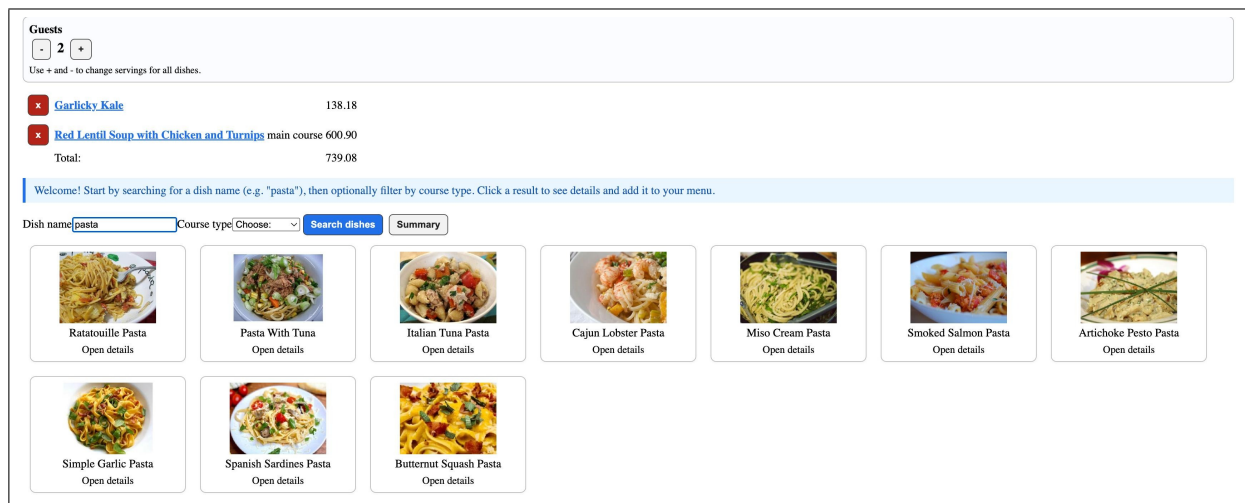
User feedback (think-aloud test). Table 2 lists the problems the user ran into and the changes made in response.

Problem observed	Change made
The search area was unclear; the user was unsure what to type or choose first.	Added a first-use hint and clear labels and placeholders to the search area.
The user couldn't easily tell which dishes were already in the menu.	Dishes already in the menu are highlighted in the search results.
The guest count controls were hard to notice.	Guest count controls are grouped, labelled and styled for visibility.
The user was unsure how to go back after opening dish details.	Added a prominent <i>Back to search</i> button and helper text to the details view.
Some buttons looked like plain text and weren't obviously clickable.	All buttons (search, summary, add/remove, back) have distinct primary, secondary and danger styles. The summary view was also restyled.

Table 2: Think-aloud findings and the changes made.



(a) Start view: hint, labelled search and grouped guest controls.



(b) Searching “pasta” with two dishes in the menu.

Figure 3: The search view after the user-feedback improvements.

Result: the app became much more user-friendly and visually clear. The user could easily understand how to search, see which dishes were in the menu, adjust the guest count and move between views (Figure 3).

What was learnt.

- *Persistence*: saving application state in the cloud and loading it again on startup.
- *Reactive objects and side effects*: a MobX **reaction** watches the model and triggers saving. The **ready** flag prevents a race between reading and writing.
- *Involving users*: a short think-aloud test found problems that were not visible to the developer. The unit tests passed, but users still struggled with the UI.

5 Bonus Tasks

- **Reactivity on persistence** (Bonus: **Reactivity on persistence**): an **onSnapshot** listener updates the model whenever the Firestore document changes. If DinnerPlanner is open in two windows side by side, a change in one appears in the other. A flag stops these incoming updates from being saved back to Firestore.
- **Search-as-you-type with debouncing** (Search-as-you-type with debouncing): every keystroke restarts a 1-second timer (**scheduleSearch**), so at most one search per second is

sent to the API.

- **Infinite search result scroll** (`Infinite search result scroll`): when the user scrolls near the bottom, `loadMoreResults` fetches the next 10 results with an increased `offset` and appends them to the list. A second request is not started while one is still pending.

What was learnt: timers as side effects, rate-limiting API calls, pagination with `offset/number`, and keeping several windows in sync through the cloud.

6 Wrap-up and Future Work

The final commits merged the user improvements into `main` and removed the embedded Firebase credentials from the repository. The configuration is now read from environment variables, so no secrets are committed.

Possible improvements:

- Render the recipe instructions as sanitised HTML. The raw `<ol><li>` tags are currently visible on the details page.
- Add user authentication so that each user has a separate menu.
- Move `searchParams` and `currentDishPromiseState` into component state (the component-state bonus tasks).
- Make the layout responsive for mobile devices.